

집합, 맵, 우선순위 큐

권용환

2026 ALOHA 초급반 1주차

1. 자료구조



- ✓ 이번 시간에 배우는 집합, 맵, 우선순위 큐는 모두 자료구조이다.
- ✓ 자료구조란 데이터를 효율적으로 저장하고 관리하는 방식이다.
- ✓ 먼저 집합에 대하여 알아보자.

2. 집합



- ✓ 수학에서 집합(Set)은 어떤 대상들의 모임이다.
- ✓ 알고리즘에서 말하는 집합도 수학에서 말하는 것과 크게 다르지 않다.
- ✓ 알고리즘에서의 집합은 같은 자료형의 변수를 모아 놓은 묶음이다.
- ✓ 이 설명을 듣다 보면 배열과 별 차이가 없는 것처럼 보인다.
- ✓ 그렇다면 배열과 집합의 차이는 무엇일까?



- ✓ 집합에 원소를 저장해 놓으면 어떤 원소가 집합에 들어 있는지 빠르게 찾을 수 있다.
- ✓ 예를 들어 배열 $[1, 2, 3, 4]$ 안에 3이 있는지 확인하려면 배열의 모든 원소를 순회해야 한다. 이는 $\mathcal{O}(N)$ 의 시간 복잡도를 가진다.
- ✓ 하지만 집합에 배열과 동일한 원소 $\{1, 2, 3, 4\}$ 를 넣어 놓으면 특정 원소가 있는지 $\mathcal{O}(\log N)$ 만에 확인할 수 있다.
- ✓ 집합이 어떻게 배열보다 빠르게 특정 원소의 존재 여부를 판정할 수 있는지 알고 싶다면 이진 탐색 트리(Binary Search Tree)를 참고하면 된다. 이진 탐색 트리는 2학년 자료구조 과목에서 배우게 된다.

2. 집합



- ✓ C++에서 집합을 사용하기 위해서는 `<set>` 을 include하면 된다.
- ✓ `set<자료형>` 처럼 작성하면 자료형 을 담는 집합을 만들 수 있다.

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s;
    // 다음 페이지에 코드가 이어진다.
```

2. 집합



- ✓ 집합에 원소 x 를 삽입하려면 `s.insert(x)` 처럼 작성하면 된다.
- ✓ 집합에서 원소 x 를 삭제하려면 `s.erase(x)` 처럼 작성하면 된다.
- ✓ 집합에 없는 원소에 `erase`를 사용해도 컴파일 에러가 나지 않는다.

```
// 이어서...  
s.insert(4); // 집합에 4 삽입  
s.insert(6); // 집합에 6 삽입  
s.insert(9); // 집합에 9 삽입  
s.erase(4); // 집합에서 4 삭제  
s.erase(8); // 8은 집합에 없지만 문제가 되지 않는다.
```



2. 집합

- ✓ 집합에 원소 x 가 있는지 확인하려면 `s.find(x)`를 활용한다.
- ✓ x 가 집합에 있다면 `s.find(x)`는 x 의 포인터를 반환한다.
- ✓ 사실 포인터가 아니고 이터레이터(iterator)이긴 하지만 포인터라고 알아둬도 큰 문제는 없다.
- ✓ x 가 집합에 없다면 `s.find(x)`는 `s.end()`를 반환한다.

```
// 이어서...  
if (s.find(6) != s.end()) {  
    cout << "6 is found in the set.\n";  
} else {  
    cout << "6 is not found in the set.\n";  
}  
}
```

- ✓ 위 코드는 `6 is found in the set.`을 출력한다.

2. 집합



- ✓ 이외에도 `s.size()` 는 현재 집합의 크기를 반환하고, `s.empty()` 는 집합이 비어 있는지 확인할 수 있다.

2. 집합



- ✓ 집합을 사용할 때 유의할 점이 몇 가지 있다.
- ✓ 첫 번째로, `set` 은 원소의 중복을 허용하지 않는다. 즉 이미 집합에 존재하는 원소를 한 번 더 삽입하면 아무 일도 일어나지 않는다.
- ✓ 사실 이 특징이 배열과 집합의 가장 큰 차이이기도 하다.
- ✓ 중복을 허용하고 싶다면 `set` 대신에 `multiset` 을 사용하면 된다.
- ✓ 단 `multiset` 을 사용할 경우에는 `erase(x)` 가 `multiset` 내의 모든 x 를 삭제한다.
- ✓ 한 개의 x 만 삭제하고 싶다면 `erase(find(x))` 를 호출하자.

2. 집합



- ✓ 따라서 아래 코드는 `s is empty`를 출력한다.

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s;
    s.insert(1); // 집합에 1을 넣는다.
    s.insert(1); // 아무 일도 일어나지 않는다.
    s.erase(1); // 집합에서 1을 삭제한다.
    if (s.empty()) {
        cout << "s is empty\n";
    }
}
```

2. 집합



- ✓ 두 번째로, set 은 원소를 오름차순으로 정렬하여 저장한다.
- ✓ 따라서 원소를 넣는 순서는 상관이 없다.
- ✓ 세 번째로, set 의 insert, erase, find 의 시간 복잡도는 모두 $\mathcal{O}(\log N)$ 이다.
- ✓ 이것은 위에서 설명했던 이진 탐색 트리의 특성 때문이다.

2. 집합



- ✓ 배열처럼 집합도 안에 들어 있는 원소를 순회할 수 있다.
- ✓ `for (int e: s)` 처럼 `for` 문을 적으면 된다. 이때 `e` 는 집합에 들어 있는 원소의 자료형과 같아야 한다.
- ✓ 아래 코드는 `int` 형 집합 `s` 의 모든 원소를 순회하면서 출력한다.

```
for (int e: s) {  
    cout << e << " ";  
}
```

2. 집합



- ✓ 집합을 활용하는 연습 문제인 '도둑 잡아라'(JUNGOL 7009)를 풀어 보자.
- ✓ N 개의 주민 번호가 주어지고 Q 개의 용의자 중 주민이 아닌 사람을 찾아 출력하는 문제이다.
- ✓ 가장 쉽게 생각할 수 있는 방법으로는 주민 번호를 모두 배열에 저장하고 각 용의자의 주민 번호마다 이 번호가 주민 번호인지 배열을 순회하면서 확인하는 방법이 있다.
- ✓ 하지만 이는 최악의 경우에 $\mathcal{O}(NQ)$ 의 시간이 걸리게 되어 너무 느리다.

2. 집합



- ✓ 이때 집합을 활용할 수 있다.
- ✓ N 개의 주민 번호를 모두 set에 넣으면 특정 번호가 주민 번호인지 $\mathcal{O}(\log N)$ 안에 판별할 수 있다.
- ✓ 따라서 전체 시간 복잡도는 $\mathcal{O}(Q \log N)$ 이 되어 제한 시간 내에 돌아가게 된다.
- ✓ 주민이 아닌 용의자가 한 명도 없는 경우를 유의하자. 주민이 아닌 용의자를 세 주면 된다.

2. 집합



```
#include <iostream>
#include <set>
using namespace std;

int main() {
    cin.tie(0)->sync_with_stdio(0);
    set<int> s;
    int n, q, cnt = 0; cin >> n >> q;
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        s.insert(x);
    }
    for (int i = 0; i < q; i++) {
        int x; cin >> x;
        if (s.find(x) == s.end()) {
            cnt++;
            cout << x << " ";
        }
    }
    if (cnt == 0) cout << "-1\n";
    return 0;
}
```

3. 맵



- ✓ 맵(Map)은 key와 value로 이루어진 자료구조이다.
- ✓ C++에서 맵을 사용하기 위해서는 <map> 을 include하면 된다.
- ✓ map<key 자료형, value 자료형> 처럼 선언할 수 있다.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int, int> m;
}
```



- ✓ 맵의 값을 변경하거나 접근하고 싶다면 배열처럼 대괄호를 이용하면 된다.

```
map<int, int> m1;  
m1[2] = 10;  
cout << m1[4] << "\n";
```

- ✓ 맵의 장점은 key로 사용할 수 있는 자료형과 값의 제한이 없다는 점이다.
- ✓ 배열은 무조건 0 이상 배열의 크기 미만의 정수 인덱스를 사용해야 한다.
- ✓ 하지만 맵은 음수, 소수, 문자열 등의 다양한 자료형을 key로 사용할 수 있다.
- ✓ key와 value를 한 쌍으로 저장한다고 생각하면 편리하다.



- ✓ 맵도 집합처럼 원소(key와 value 쌍)들을 key의 오름차순으로 정렬하여 저장한다.
- ✓ 맵에서 특정 원소를 접근하려면 $\mathcal{O}(\log N)$ 의 시간이 걸린다.
- ✓ 맵의 가장 대표적인 활용처는 빈도 카운팅이다. 맵[원소] 값을 특정 원소가 등장한 빈도(횟수)로 정의하는 것이다.



- ✓ 맵의 원소를 순회할 때에 for 문을 이용할 수 있다. 여기서 k는 key, v는 value이다.

```
for (auto [k, v]: m) {  
    cout << k << " " << v << "\n";  
}
```

4. 우선순위 큐



- ✓ 배열에서 최댓값을 찾는 문제를 생각해 보자.
- ✓ 배열 전체를 순회하면 $\mathcal{O}(N)$ 에 해결할 수 있다.
- ✓ 만약 최댓값을 찾고 삭제하는 연산을 N 번 해야 한다면 어떨까?
- ✓ 배열 전체를 순회하면서 최댓값을 찾고 삭제한다면 $\mathcal{O}(N^2)$ 에 해결할 수 있다.
- ✓ 배열을 미리 정렬해 둔다면 더 빠른 $\mathcal{O}(N \log N)$ 에 해결할 수도 있다.

4. 우선순위 큐



- ✓ 연산 중간에 새로운 원소를 추가한다면 어떨까?
- ✓ 배열을 미리 정렬해 두는 방법을 그대로 사용해 보자.
- ✓ 새 원소를 추가할 때에는 배열의 적당한 위치에 해당 원소를 삽입해야 한다.
- ✓ 이렇게 한다면 새 원소를 추가할 때 배열 전체를 순회해야 하므로 시간 복잡도가 $\mathcal{O}(N)$ 이 되고 전체 시간 복잡도는 $\mathcal{O}(N^2)$ 이 된다.
- ✓ 이는 배열을 정렬하지 않고 매번 최댓값을 찾는 것과 별반 다를 바가 없다.
- ✓ 더 빠른 방법은 없을까?

4. 우선순위 큐



- ✓ 이 연산을 빠르게 할 수 있게 만드는 자료구조가 우선순위 큐(Priority Queue)이다.
- ✓ 요소의 삽입과 삭제가 모두 $O(\log N)$ 에 가능하다.
- ✓ C++에서는 기본적으로 최댓값을 꺼낼 수 있는 우선순위 큐를 지원한다. 따라서 이 강의에서는 우선순위 큐가 최댓값을 꺼낸다고 가정한다.
- ✓ 물론 최댓값 대신 최솟값을 꺼낼 수도 있고, 원하는 정렬 기준을 선택하여 값을 꺼낼 수도 있다.

4. 우선순위 큐



- ✓ C++에서 우선순위 큐를 사용하려면 `<queue>` 와 `<vector>` 를 include하자.
- ✓ `priority_queue<자료형>` 으로 자료형을 담는 우선순위 큐를 만들 수 있다.

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

int main() {
    priority_queue<int> pq;
}
```

4. 우선순위 큐



- ✓ 우선순위 큐에 원소 x 를 삽입하려면 `pq.push(x)` 처럼 작성하면 된다.
- ✓ 우선순위 큐에서 최댓값을 확인하려면 `pq.top()` 을 활용하면 된다.
- ✓ 우선순위 큐에서 최댓값을 꺼내려면 `pq.pop()` 을 활용하면 된다.

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

int main() {
    priority_queue<int> pq;
}
```



4. 우선순위 큐

✓ 아래 코드를 살펴보자.

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

int main() {
    priority_queue<int> pq;
    pq.push(5);
    pq.push(4);
    pq.push(6);
    cout << pq.top() << "\n"; // 6
    pq.pop();
    cout << pq.top() << "\n"; // 5
}
```

4. 우선순위 큐



- ✓ 이외에도 집합과 비슷하게 `pq.size()` 는 우선순위 큐의 원소 개수를 반환하고 `pq.empty()` 로 우선순위 큐가 비어 있는지 확인할 수 있다.

4. 우선순위 큐



- ✓ 최솟값을 꺼낼 수 있는 우선순위 큐를 만드려면 아래와 같이 하면 된다.

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

int main() {
    priority_queue<int> pq_max; // 최댓값을 꺼내는 우선순위 큐
    priority_queue<int, vector<int>, greater<int>> pq_min; // 최솟값을 꺼내
    는 우선순위 큐
}
```

5. 연습 문제



- ✓ **JUNGOL 7009** 도둑 잡아라
앞에서 해설한 문제이다.
- ✓ **JUNGOL 4637** Tutorial: STL Set 1
set 을 사용해 보자.
- ✓ **JUNGOL 3142** ID검사
set<string> 을 활용하여 등록한 사람과 로그인한 사람을 관리하자.
- ✓ **JUNGOL 8036** 평행하는 선분의 수
특정 x좌표와 y좌표가 등장한 횟수를 map 으로 관리하자.

5. 연습 문제



- ✓ **JUNGOL 3706** 합이 0이 되는 연속구간 세기
map 을 활용하여 누적 합의 빈도를 관리하자.
- ✓ **JUNGOL 1190** 모두더하기
각 단계마다 어떤 수를 더해야 총 비용이 최소일까?